

Practical No.:	06
Practical Title :	To understand Docker Architecture and Container Life Cycle, install Docker and execute docker commands to manage images and interact with containers
Aim:	To understand Docker architecture and container lifecycle by installing Docker and executing Docker commands.
Objective:	To understand the basic architecture of Docker including client, daemon, and registry. To install and configure Docker on the system. To learn how to pull and manage Docker images from repositories like Docker Hub. To create, run, stop, and remove Docker containers.

Theory:

Docker is a platform for developing, shipping, and running applications in lightweight containers. It ensures consistency across multiple development and production environments.

Docker Architecture Components:

1. **Docker Engine:** The core part of Docker responsible for running containers.
2. **Docker Images:** Read-only templates used to create containers.
3. **Docker Containers:** Running instances of Docker images.
4. **Docker Registry:** Stores Docker images (e.g., Docker Hub, private registry).
5. **Docker Daemon:** Runs in the background and manages containers.
6. **Docker CLI:** Command-line interface to interact with Docker.

Container Lifecycle:

1. **Create:** Create a new container from an image.
2. **Start:** Start the created container.
3. **Pause/Unpause:** Pause or resume a running container.
4. **Stop:** Gracefully stop a running container.
5. **Kill:** Forcefully stop a container.
6. **Restart:** Stop and start the container again.
7. **Remove:** Delete a stopped container.

Docker is being widely used in the information technology world. It is probably one of the most used buzzwords in the past few years. With the introduction of DevOps, Docker's significance has only grown since it comes with some great features. With great features, new threats get introduced. Docker is commonly used by development and operations teams in many large organizations. If you are serious about your organization's security, it is important to understand that a simple Docker misconfiguration can lead to serious damage to your infrastructure as well as the organization. So how do we ensure that it is safely used in production as well as other non- production environments? This is where we are bringing this book to you to give you the fundamental Docker security knowledge.

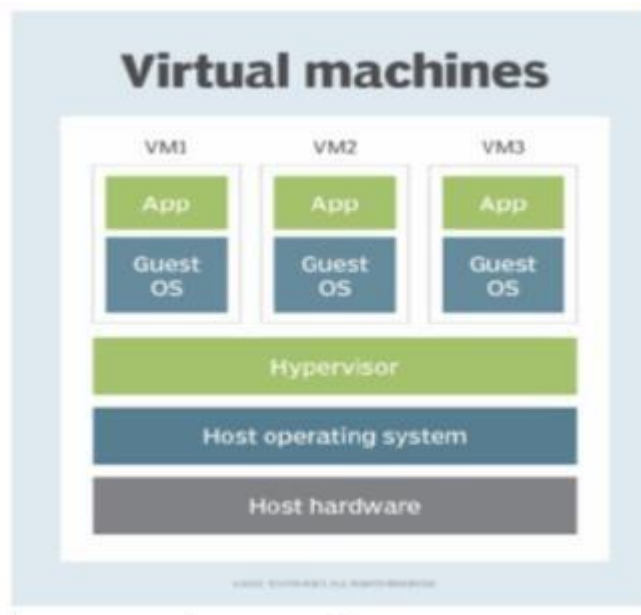
This book covers several misconfigurations and practical attacks that are possible within the Docker ecosystem.

By the end of this Practical, you will have learned Protecting Docker Environments

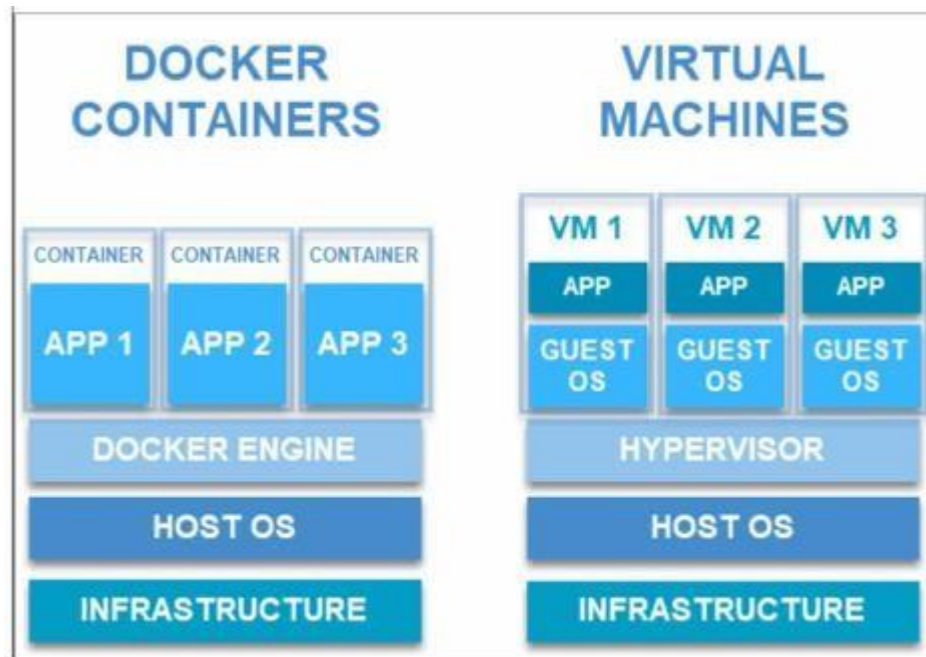
- Fundamentals of Docker
- Hacking Docker containers
- Automated vulnerability assessments
- Protecting Docker Environments



Virtual machines vs containers



Virtualization makes use of hypervisors to create separate operating system environments. Each virtual machine acts as a separate computer and it will have its own operating system whereas Docker containers make use of the host operating system's Linux kernel; which means your containers do not need a separate operating system to run on. They can make use of your host machine's Linux kernel. As you can see in the figure, Virtual machines on the left have the hardware to start with, on top of which we have the host operating system. There is a hypervisor on top of your operating system. On top of the operating system, there are different virtual machines running and each Virtual machine has a guest operating system running inside. This means each Virtual machine contains a separate operating system which makes the virtual machine's size gigantic.



In contrast, if you take a look at the containers you have the hardware, you have a host operating system running on it, and Docker engine is installed on the host operating system to act as a layer between your host operating system and your containers. If you look at the containers, they do not have any separate operating systems like what we have with virtual machines. This is the fundamental difference between Virtual machines and containers. One big advantage of Docker is the size of containers. This is possible because containers do not need to use a separate operating system for each container. We can have any Linux based operating system as the host operating system and we will still be able to layer other operating systems on top of the host. For example, let's assume that the host operating system is Ubuntu. Regardless of what operating system is running on the host, we can have containers running with CentOS, Red Hat, etc.

Executing Docker Commands

1. Checking Docker Status

`docker info`
`docker version`

2. Managing Docker Images

- **List available images:**
`docker images`
- **Pull an image from Docker Hub:**
`docker pull ubuntu`
- **Remove an image:**
`docker rmi ubuntu`

3. Working with Containers

- **Run a container:**
`docker run -it ubuntu bash`
- **List running containers:**
`docker ps`
- **List all containers (including stopped ones):**
`docker ps -a`

- **Stop a running container:**
docker stop <container_id>
- **Restart a container:**
docker restart <container_id>
- **Remove a container:**
docker rm <container_id>

4. Checking Logs of a Container

docker logs <container_id>

5. Executing Commands inside a Running Container

docker exec -it <container_id> bash

Conclusion:
